Certificate Management System v1.3

Project Submission Report

Submission Date:	18 September 2026
Project Type:	Department Certificate & OD Management Web Application
Version:	v1.3
Status:	Implemented & Locally Validated
Backend:	Node.js · Express.js · PostgreSQL 18
Frontend:	React 18 · Vite · Tailwind CSS
Authentication:	JWT (HS256) · bcrypt

Table of Contents

1. Executive Summary
2. Project Objectives
3. Technology Stack
4. System Architecture
5. Role Hierarchy & Authentication
6. User Workflows
7. Database Design
8. API Reference
9. File Storage & Report Generation
10. Security & Validation
11. Local Setup & Configuration
12. Test Credentials
13. Project Structure
14. Development Status & Validation
15. Pending Department Decisions
16. Out-of-Scope Features

1. Executive Summary

The Certificate Management System is a full-stack web application for a college department that replaces fragmented spreadsheet and file-sharing workflows with a centralised, role-aware platform backed by PostgreSQL. It manages the complete lifecycle of student On-Duty (OD) participation: from initial registration by a Student Representative, through certificate submission by the Student, to monthly reporting and bulk export by Coordinating Faculty.

The system enforces strict ownership — a student can only view and submit certificates for their own OD records. Every API endpoint enforces server-side role checks so the backend cannot be bypassed by manipulating the frontend. Registration numbers (OD-0001, OD-0002, …) are generated atomically via a PostgreSQL sequence, guaranteeing uniqueness under concurrent use.

> The system was validated end-to-end on 18 September 2026 against a live PostgreSQL 18 instance. All three migrations applied cleanly, both test suites passed (5 tests total), and the complete OD-creation !’ certificate-upload !’ Excel/ZIP-export workflow was confirmed.

2. Project Objectives

- Provide a central, searchable record for all OD registrations within a department.
- Generate globally unique OD registration numbers safely under concurrent use via PostgreSQL sequences.
- Allow students to submit digital certificates (PDF, JPG, JPEG, PNG "d 5 MB) against their own OD records.
- Prevent any user from viewing or modifying another student's records (ownership enforcement).
- Provide coordinating faculty with monthly summaries, Excel exports, and certificate ZIP downloads.
- Store certificate files in a predictable, protected filesystem structure — never served directly over HTTP.
- Support a four-role hierarchy (Admin !’ Faculty !’ Student Rep !’ Student) with provisioning workflows.
- Enforce all access control server-side so the backend cannot be bypassed by client manipulation.

3. Technology Stack

Backend

Package / Technology	Version	Purpose
Node.js	LTS v24	JavaScript runtime (ES Modules)
Express.js	^4.18.2	HTTP web framework
PostgreSQL	12+ (tested on 18)	Relational database
pg	^8.11.3	PostgreSQL connection pool driver
jsonwebtoken	^9.0.2	JWT signing & verification (HS256)
bcrypt	^5.1.1	Password hashing (cost factor 12)
multer	^1.4.5-lts.1	Multipart file upload (memory buffer)

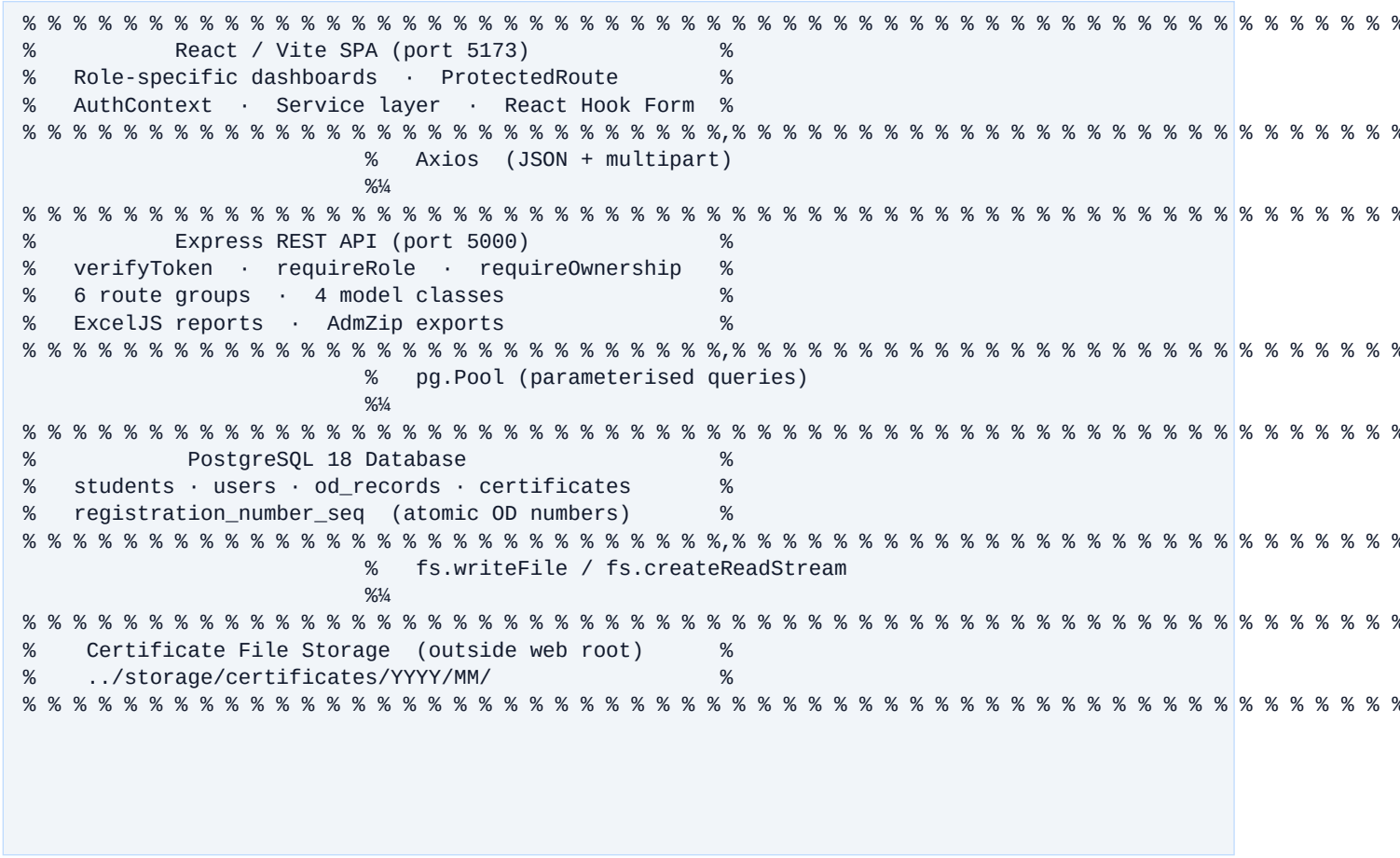
express-validator	^7.0.0	Request input validation
exceljs	^4.3.0	Excel .xlsx report generation
adm-zip	^0.5.10	ZIP archive creation for bulk downloads
cors	^2.8.5	Cross-Origin Resource Sharing headers
dotenv	^16.3.1	Environment variable loading
jest + supertest	^29 + ^6.3.3	Backend unit & integration testing
nodemon	^3.0.2	Dev auto-restart on file changes

Frontend

Package / Technology	Version	Purpose
React	^18.2.0	UI component framework
Vite	^5.0.8	Build tool & HMR dev server
React Router DOM	^6.20.1	Client-side routing & navigation
Axios	^1.6.2	HTTP client with request/response interceptors
React Hook Form	^7.48.0	Form state management & validation
Tailwind CSS	^3.3.6	Utility-first CSS framework
Vitest	^2.1.8	Vite-native test runner
Testing Library	^16.0.1	React component testing utilities
ESLint + Prettier	^8 + ^3.1.0	Static analysis & code formatting

4. System Architecture

The application uses a layered client-server architecture. The React SPA communicates with the Express REST API via Axios over HTTP/HTTPS using JSON for data and multipart/form-data for file uploads. The API layer handles authentication, authorisation, business logic, and database access. Certificate files are persisted to the local filesystem outside the web root.



Design Principles

- Separation of concerns — routing, middleware, models, and config are in distinct layers.
- Fail-fast production guards — the app refuses to start with insecure configuration.
- Parameterised SQL everywhere — no string interpolation; eliminates SQL injection.
- Ownership at the data layer — student ownership checked in models, not just routes.
- Stateless authentication — JWTs; no server-side session store required.
- Gap-tolerant sequence — registration numbers may have gaps but are never reused.

5. Role Hierarchy & Authentication

Role Hierarchy

```
admin
  coordinating_faculty   (provisioned by admin)
    student_representative  (provisioned by faculty)
student   (linked to students table via student_id FK)
```

Role Capabilities

Role	DB Value	Created By	Key Capabilities
Administrator	admin	bootstrap-admin script	Provision & manage coordinating faculty accounts

Coordinating Faculty	coordinating_faculty	Admin	View all OD records & certs; generate Excel/ZIP reports; manage reps
Student Representative	student_representative	Faculty	Search students; register OD records; view OD list
Student	student	Seeded from students DB	View own profile & OD records; upload & download own certificates

Authentication Flow

- Students authenticate with their institutional register number as the username and their date of birth in YYYY-MM-DD format as the password.
- Staff (faculty, rep, admin) authenticate with a provisioned username and bcrypt-hashed password (cost factor 12).
- On success the server signs a JWT { user_id } with HS256 and returns { token, user }. Token lifetime defaults to 30 minutes (JWT_EXPIRY).
- All protected routes run verifyToken: extracts the Bearer token, verifies the signature, loads the user from the database, and checks is_active = true.
- requireRole(...allowedRoles) checks req.user.role against the allowed list.
- requireStudentOwnership compares req.user.student_id with the resource's student_id.
- Logout is stateless: the server returns 200 and the client discards the token.
- The frontend Axios interceptor auto-logs out and redirects to /login on any 401 response.

6. User Workflows

Student Workflow

1. Log in with institutional register number and date of birth (YYYY-MM-DD).
2. View personal profile: name, department, year, section.
3. View list of own OD records with certificate submission status per record.
4. Select an OD registration number that does not yet have a certificate.
5. Upload a PDF, JPG, JPEG, or PNG certificate file up to 5 MB.
6. View or download previously submitted certificates.

Student Representative Workflow

1. Log in with a provisioned staff credential.
2. Search students by register number, name, or department.
3. Select the target student from search results.
4. Fill in event details and submit the OD registration form.
5. Receive the system-generated registration number (e.g., OD-0042).
6. View the full list of OD records with their certificate status.

Coordinating Faculty Workflow

1. Log in with a provisioned faculty credential.

2. Select a reporting month (YYYY-MM) using the month picker.
3. View the monthly summary: total OD count, submitted certificates, and missing certificates.
4. Browse the full OD records table with optional department/year/section filters.
5. Review the missing certificates list to identify students who have not submitted.
6. Download the filtered records as a formatted Excel (.xlsx) spreadsheet.
7. Download all submitted certificates for the month as a ZIP archive.

Administrator Workflow

1. Log in with the administrator credential.
2. View all coordinating faculty accounts with active/inactive status.
3. Create new coordinating faculty accounts (username + password).
4. Activate or deactivate existing faculty accounts.

7. Database Design

Migrations

File	Contents
001_initial_schema.sql	Creates students, users, od_records, certificates tables; registration_number_seq sequence; all indexes and foreign keys.
002_authentication_and_role_hierarchy.sql	Adds password_hash and assigned_faculty_id to users; adds role CHECK constraint; sets student usernames; adds auth indexes.
003_admin_role_and_student_dob.sql	Adds admin to role CHECK; adds date_of_birth DATE column to students; adds idx_students_date_of_birth index.

Entity Relationship

```
students (1) % % % % % % % % % % % % % < od_records (1) % % 0..1% %  certificates
    %                              %
    %   student_id FK              %   od_id UNIQUE FK
    %                              %
users % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %   (student_id nullable for staff)
users (assigned_faculty_id) % % % % % % > users  (rep !’ faculty self-ref)
```

Table: students

Column	Type	Constraints	Description
student_id	BIGSERIAL	PRIMARY KEY	Internal system identifier
register_number	VARCHAR(20)	NOT NULL, UNIQUE	Institutional college number e.g. 22AD001
name	VARCHAR(255)	NOT NULL	Student full name

department	VARCHAR(100)	NOT NULL	Department name
year	VARCHAR(50)		Academic year e.g. "3rd Year"
section	VARCHAR(50)		Class section
date_of_birth	DATE		Used as student login password
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time
updated_at	TIMESTAMP	DEFAULT NOW()	Last update time

Table: users

Column	Type	Constraints	Description
user_id	BIGSERIAL	PRIMARY KEY	Internal user identifier
student_id	BIGINT	FK !' students, NULLABLE	NULL for staff users
username	VARCHAR(100)		Login username
role	VARCHAR(50)	NOT NULL, CHECK	admin | coordinating_faculty | student_representative | student
password_hash	TEXT		bcrypt hash of password
is_active	BOOLEAN	DEFAULT TRUE	Account active flag
assigned_faculty_id	BIGINT	FK !' users(user_id)	Rep's assigned faculty (NULL for non-reps)
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time

Table: od_records

Column	Type	Constraints	Description
od_id	BIGSERIAL	PRIMARY KEY	Internal OD identifier
student_id	BIGINT	FK !' students, NOT NULL	Student who received OD
registration_number	VARCHAR(20)	UNIQUE, NOT NULL	System-generated e.g. OD-0001
event_details	TEXT		Event name and description
created_by	VARCHAR(100)		Username of rep who registered
od_status	VARCHAR(50)	DEFAULT 'active'	active | cancelled
created_at	TIMESTAMP	DEFAULT NOW()	Registration timestamp

Table: certificates

Column	Type	Constraints	Description
certificate_id	BIGSERIAL	PRIMARY KEY	Internal cert identifier

od_id	BIGINT	UNIQUE FK !' od_records	One certificate per OD (enforced by UNIQUE)
student_id	BIGINT	FK !' students, NOT NULL	Student who submitted
file_name	VARCHAR(255)	NOT NULL	Display filename: reg_OD-NNNN.ext
file_path	VARCHAR(500)	NOT NULL	Absolute disk path — never in API responses
file_size	INTEGER		Size in bytes
file_type	VARCHAR(20)		pdf | jpg | jpeg | png
submission_month	VARCHAR(7)		YYYY-MM — used for report filtering
submitted_at	TIMESTAMP	DEFAULT NOW()	Upload timestamp

Sequence: registration_number_seq

- PostgreSQL SEQUENCE starting at 1, incrementing by 1.
- Consumed inside a transaction in ODRecord.create() — rolled back on error.
- Gaps may occur after failed transactions — this is expected and does not compromise uniqueness.
- Formatted as OD-{value padded to 4 digits}: value 1 !' OD-0001, value 42 !' OD-0042.
- Values are never reused, reassigned, or recycled.

8. API Reference

Base URL: http://localhost:5000/api | All protected endpoints require: Authorization: Bearer <JWT>

Health

Method	Path	Auth	Description
GET	/health	None	Returns { status: "OK", timestamp }

Authentication — /auth

Method	Path	Auth	Role	Description
POST	/auth/login	No	—	Login. Body: { username, password }. Returns { token, user }.
POST	/auth/logout	JWT	Any	Stateless logout. Returns { message }.
GET	/auth/me	JWT	Any	Returns full user object + linked student info.

User Management — /users

Method	Path	Role	Description
GET	/users/coordinating-faculty	admin	List all faculty accounts
POST	/users/coordinating-faculty	admin	Create faculty. Body: { username, password }
PATCH	/users/coordinating-faculty/:userId	admin	Update faculty is_active or password
GET	/users/student-representatives	coordinating_faculty	List reps assigned to calling faculty
POST	/users/student-representatives	coordinating_faculty	Create rep (auto-assigned to caller). Body: { username, password }
PATCH	/users/student-representatives/:id	coordinating_faculty	Update own rep's is_active or password

Students — /students

Method	Path	Role	Description
GET	/students/me	student	Authenticated student's profile
GET	/students/me/od-records	student	Own OD records. Query: ?limit&offset
GET	/students/me/certificates	student	Own submitted certificates. Query: ?limit&offset
GET	/students	student_representative	Search students. Required: ?q=. Optional: ?limit&offset

OD Records — /od-records

Method	Path	Role	Description
POST	/od-records	student_representative	Create OD. Body: { student_id, event_details }. Returns registration_number.
GET	/od-records	student_representative / coordinating_faculty	List records. Filters: ?department&status&limit&offset
GET	/od-records/:odId	student / rep / faculty	Single record + cert metadata. Students restricted to own records.

Certificates — /certificates

Method	Path	Role	Description

POST	/certificates	student	Upload cert. Multipart: registration_number + file (PDF/ JPG/JPEG/PNG "d 5 MB). Enforces ownership & 1-per-OD.
GET	/certificates	coordinating_faculty	List certs. Filters: ?student_id&submission_month&department&year&section&limit&offset
GET	/certificates/:certId/download	student (own) / faculty (any)	Stream file download with correct Content-Type.

Reports — /reports

Method	Path	Role	Description
GET	/reports/monthly-summary	coordinating_faculty	JSON summary. Query: ?month=YYYY-MM&department?&year?&section?. Returns counts + records + missing list.
POST	/reports/excel	coordinating_faculty	Download .xlsx. Body: { month, department?, year?, section? }.
POST	/reports/certificates-zip	coordinating_faculty	Download .zip of certs for month. Body: { month, department?, year?, section? }. Missing files skipped.

Error Response Format

```
{
  "error": {
    "code":    "NOT_FOUND",
    "message": "OD record not found"
  }
}
```

Common status codes: 400 (validation) · 401 (unauthenticated) · 403 (forbidden / wrong role) · 404 (not found) · 409 (conflict — duplicate certificate) · 413 (file too large) · 500 (server error).

9. File Storage & Report Generation

Certificate File Storage

Multer buffers the uploaded file in memory. After validation the route handler in routes/certificates.js writes it to disk under STORAGE_PATH.

```
STORAGE_PATH/
% % %  YYYY/
    % % %  MM/
        % % %  {register_number}_{registration_number}.{ext}

Example: ../storage/certificates/2026/09/22AD001_OD-0001.pdf
```

- file_path in the certificates table stores the absolute disk path — never returned raw in API responses.
- File access goes exclusively through GET /certificates/:certId/download after ownership/role checks.
- Allowed file types: PDF, JPG, JPEG, PNG. Maximum size: 5 MB (MAX_FILE_SIZE env var).
- The storage directory is outside the frontend public/ directory — no direct HTTP access is possible.

Excel Report Generation

- Uses ExcelJS to build an in-memory workbook with two sheets.
- Sheet 1: All OD records matching the month/department/year/section filters, with student details and certificate status.
- Sheet 2: Students who received OD but have not submitted a certificate.
- Returned as application/vnd.openxmlformats-officedocument.spreadsheetml.sheet with a timestamped filename.

ZIP Report Generation

- Uses adm-zip to bundle all certificate files returned by Certificate.getByMonth().
- Each file is added to the ZIP archive using its stored filename.
- Files missing from disk are silently skipped and logged server-side.
- Returned as application/zip.

10. Security & Validation

Security Control	Implementation
JWT Authentication	HS256 JWT; verified on every protected request; user re-loaded from DB to check is_active.
Role-Based Access Control	requireRole() middleware enforces allowed roles server-side on every endpoint.
Student Ownership	requireStudentOwnership() compares req.user.student_id with the resource's student_id.
Password Storage	bcrypt hash with cost factor 12; plaintext passwords are never stored.
SQL Injection Prevention	All queries use parameterised placeholders via pg — no string concatenation.
File Type Validation	Multer fileFilter + server-side MIME check; only PDF, JPG, JPEG, PNG accepted.
File Size Limit	Multer limits enforced at 5 MB (MAX_FILE_SIZE). Returns 413 on excess.
Secure File Storage	Files stored outside web root; served only through authenticated download endpoint.

HTTPS Enforcement (prod)	HTTP requests receive 403 in production; trust proxy enabled.
JWT Secret Strength (prod)	App refuses to start if JWT_SECRET is < 32 chars or is the placeholder value.
CORS Allowlist (prod)	Wildcard (*) rejected in production; only explicit origins allowed.
Duplicate Certificate Prevention	UNIQUE constraint on certificates.od_id; API returns 409 on duplicate upload.

11. Local Setup & Configuration

Prerequisites

- Node.js LTS (v16 or later)
- PostgreSQL 12+ (validated on PostgreSQL 18)
- npm (bundled with Node.js)

Backend Setup

```
# 1. Install dependencies
cd backend
npm install

# 2. Configure environment
copy .env.example .env
# Edit .env: DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, JWT_SECRET

# 3. Create the PostgreSQL database
# Run in psql: CREATE DATABASE certificate_management;

# 4. Run all three migrations
npm run migrate

# 5. (Optional) Create first admin account
npm run bootstrap-admin

# 6. Start the server
npm start          # production
npm run dev        # development with auto-reload
```

Backend Environment Variables

Variable	Default	Required	Description
DB_HOST	localhost	Yes	PostgreSQL hostname
DB_PORT	5432	Yes	PostgreSQL port
DB_NAME	certificate_management	Yes	Database name

DB_USER	postgres	Yes	Database user
DB_PASSWORD	—	Yes	Database password
PORT	5000	No	Express listen port
NODE_ENV	development	No	development or production
JWT_SECRET	—	Yes	HS256 signing key (min 32 chars in prod)
JWT_EXPIRY	30m	No	Token lifetime
STORAGE_PATH	../storage/certificates	Prod only	Certificate file root directory
MAX_FILE_SIZE	5242880	No	Max upload bytes (5 MB default)
CORS_ORIGIN	http://localhost:5173	Prod only	Allowed origin(s), comma-separated

Frontend Setup

```
# 1. Install dependencies
cd frontend
npm install

# 2. Configure environment
copy .env.example .env
# Set: VITE_API_BASE_URL=http://localhost:5000/api

# 3. Start development server
npm run dev -- --host localhost

# 4. Build for production
npm run build
```

Frontend Environment Variables

Variable	Example	Description
VITE_API_BASE_URL	http://localhost:5000/api	Backend API base URL — must be prefixed VITE_
VITE_ENV	development	Environment label (optional)

12. Test Credentials

> These accounts are for local development and demonstration only. Change or remove all credentials before any production deployment.

Role	Username	Password	Notes

Administrator	admin	Admin1234	Created by bootstrap-admin script
Coordinating Faculty	faculty1	Faculty123	Provisioned by admin
Student Representative	rep1	Rep12345	Provisioned by faculty; assigned_faculty_id set
Student	22AD001	2004-06-15	Login: register number + date of birth (YYYY-MM-DD)

13. Project Structure

```
certificate-management-system/
% % %  backend/
%    % % %  server.js                  # HTTP entry point
%    % % %  src/
%    %    % % %  app.js                 # Express app: CORS, routes, error handler
%    %    % % %  config/                # database.js (pg.Pool) · storage.js
%    %    % % %  middleware/            # auth.js · authorization.js · errorHandler.js
%    %    % % %  models/                # User.js · Student.js · ODRecord.js · Certificate.js
%    %    % % %  routes/                # auth · users · students · od-records · certs · reports
%    %    % % %  migrations/            # 001_initial · 002_auth_roles · 003_admin_dob · runner
%    %    % % %  utils/                 # asyncHandler · bootstrap-admin · seedTestData
%    % % %  tests/                     # auth-middleware · role-validation · uat-workflow
% % %  frontend/
%    % % %  src/
%    %    % % %  main.jsx               # React DOM entry, BrowserRouter
%    %    % % %  App.jsx                # Route definitions
%    %    % % %  context/               # AuthContext.jsx
%    %    % % %  pages/                 # Login · Student · StudentRep · Faculty · Admin · 404 · 403
%    %    % % %  components/            # Auth · Student · StudentRep · Faculty · Common · Layout
%    %    % % %  services/              # api · auth · student · od · cert · report · user
%    %    % % %  styles/                # index.css (Tailwind)
%    % % %  dist/                      # Production build output
% % %  docs/                          # API.md · DATABASE.md · DESIGN.md · PROJECT_SUBMISSION.md
% % %  storage/certificates/          # Uploaded certificate files (YYYY/MM/)
% % %  README.md
```

14. Development Status & Validation Evidence

Development Phases

- Phase 1: Project setup, database schema, and tooling
- Phase 2: Authentication framework: JWT, bcrypt, verifyToken, requireRole
- Phase 3: Student APIs: profile, OD records, certificate upload and download
- Phase 4: Student Representative APIs: student search, OD registration
- Phase 5: Coordinating Faculty APIs: records view, monthly summary, Excel/ZIP exports
- Phase 6: Frontend authentication, routing, AuthContext, ProtectedRoute
- Phase 7: Frontend student interface: profile, OD list, upload, download
- Phase 8: Frontend student representative interface: search, register, view

' Phase 9: Frontend coordinating faculty interface: records, reports, downloads
' Phase 10: Testing, end-to-end validation, and production build

Validation Evidence (18 September 2026)

- PostgreSQL 18 service running; certificate_management database reachable.
- All three database migrations completed successfully without errors.
- GET /api/health returned { status: "OK" }.
- Admin login + /auth/me flow succeeded; token verified.
- Student login (22AD001 / 2004-06-15) + /auth/me flow succeeded.
- Student Representative login (rep1 / Rep12345) succeeded.
- Coordinating Faculty login (faculty1 / Faculty123) succeeded.
- Representative created OD record; system returned registration_number: "OD-0001".
- Student uploaded 22AD001_OD-0001.pdf successfully; certificate_id returned.
- Faculty monthly summary returned: 1 OD record, 1 submitted certificate, 0 missing.
- Excel export returned valid .xlsx response with non-zero payload.
- ZIP export returned valid .zip response with non-zero payload containing the certificate.
- Backend test suite: 2 suites, 3 tests — all passed.
- Frontend test suite: 1 suite, 3 tests — all passed.
- Frontend production build (npm run build) completed without errors.

15. Pending Department Decisions

The following items require department confirmation before implementation. Placeholder structures exist in the codebase to accommodate each decision.

Item	Options / Notes	Current State
Authentication Method	Password, DOB (current default), OTP, SSO, Institutional OAuth	DOB-based login implemented; abstract provider pattern planned
OD Blocking Policy	Hard Stop (block new OD if cert missing), Soft Warning, Independent	No blocking implemented; od_status field exists for future use
OD Cancellation Authority	Who can cancel? What reasons? Show in reports?	od_status column exists; cancel UI not implemented
Late Submission Deadline	What is the deadline? How calculated? Consequences?	submission_month column exists; enforcement logic not implemented
Department Scope	Single department or multi-department system?	Department fields present; no hard tenant restrictions
Edit Authority	Can reps edit OD details? Can faculty correct mistakes? Audit log?	OD records immutable after creation; edit endpoints not created

16. Out-of-Scope Features

The following features were explicitly excluded from the current version to keep the system focused and avoid premature decisions on unconfirmed requirements.

Feature	Reason Excluded

Event announcement / discovery system	No requirement for students to discover events — OD records are created by reps.
Certificate verification / approval / rejection	No approval workflow requested; faculty reporting is read-only.
Automatic OD blocking for pending certificates	OD blocking policy is a pending department decision.
Participation verification	Verification mechanism not defined by department.
Multi-tenancy / multi-department isolation	Scope confirmed as a single-department deployment.
AI, OCR, QR codes, blockchain, GPS	No business requirement identified for these technologies.
Payments, leaderboards, gamification	Not relevant to the academic certificate management domain.
Real-time notifications / WebSockets	Not requested; polling or page refresh is sufficient.